

gingerBill

[Home](#) | [Articles](#) | [Odin](#) | [Subscribe](#)

Memory Allocation Strategies - Part 3

Stack Allocators

Series: [Memory](#) [Allocation](#)
[Strategies](#)

2019-02-15

Stack-Like (LIFO) Allocation

In the previous article, we looked at the [linear/arena allocator](#), which is the simplest of all memory allocators. In this article, I will cover the fixed-sized stack-like allocator. Throughout this article, I will refer to this allocator as a *stack allocator*.

Note: A stack-like allocator means that the allocator acts like a data structure following the *last-in*,

first-out (LIFO) principle. This has nothing to do with *the stack* or *the stack frame*.

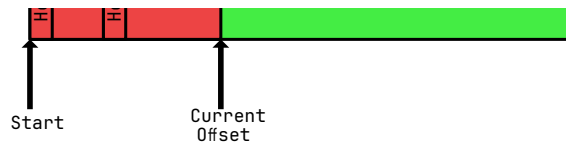
The stack allocator is the natural evolution from the arena allocator. The approach with the stack allocator is to manage memory in a stack-like fashion following the LIFO principle. Therefore, like a stack of books, if you put something on the top of the stack earlier, you need to pick that book up first before you get one underneath.

As with the arena allocator, an offset into the memory block will be stored and will be moved forwards on every allocation. The difference is that the offset can also be moved backwards when memory is *freed*. With an arena, you could only free all the memory all at once.

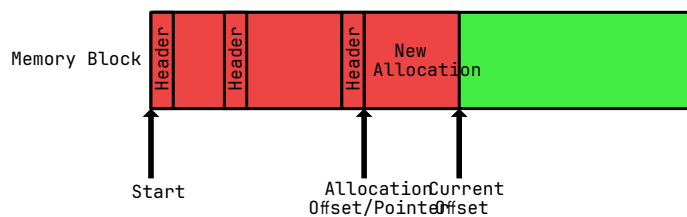
Basic Logic

As with the extended arena in the [previous article](#), the offset of the previous allocation needs to be tracked. This is required in order to free memory on a *per-allocation* basis. One approach is to store a *header* which stores information about that allocation. This *header* means that the allocator can know how far back it should move the offset to free that memory.

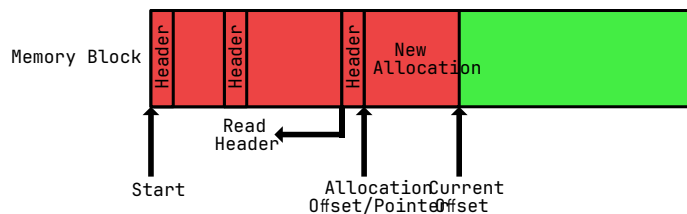




To allocate some memory from the stack allocator, as with the arena allocator, it is as simple as moving the offset forward whilst accounting for the header. In **Big-O notation**, the allocation has complexity of $O(1)$ (constant).



To free a block, the header that is stored before the block of memory can be read in order to move the offset backwards. In Big-O notation, the freeing of this memory has complexity of $O(1)$ (constant).



Header Storage

You may have noticed that I never actually state what to store in the allocation header. The reason for this is because there are numerous approaches to stack allocators which store different data. There are three main approaches

- Store the padding from the previous offset
- Store the previous offset
- Store the size of the allocation

In this article, I will cover the first two approaches, the first of which I will refer to as a *loose stack* or *small stack* as it stores very little information in the header. The third approach is useful if you want to query the size of an allocation dynamically¹.

¹ I rarely need this as I usually track the size of the allocation manually.

Implementation of the Loose/Small Stack Allocator

The stack allocator will act like an arena allocator in many regards except for the ability to free memory on a per-allocation basis. The complete stack allocator will have the following procedures:

- `stack_init` initialize the stack with a pre-allocated memory buffer
- `stack_alloc` increments the offset to indicate the current buffer offset whilst taking into account the allocation header
- `stack_free` frees the memory passed to it and decrements the offset to *free* that memory

- `stack_resize` first checks to see if the allocation being resized was the previously performed allocation and if so, the same pointer will be returned and the buffer offset is changed. Otherwise, `stack_alloc` will be called instead.
- `stack_free_all` is used to free all the memory within the allocator by setting the buffer offsets to zero.

Data Structures

The (loose/small) stack data structure contains the same information as an arena.

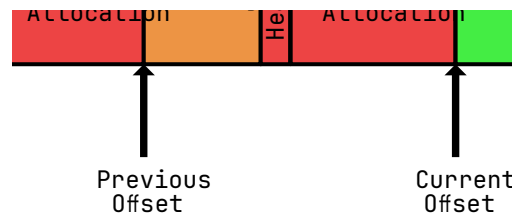
```
typedef struct Stack Stack;
struct Stack {
    unsigned char *buf;
    size_t buf_len;
    size_t offset;
};
```

The allocation header for this particular stack implementation uses an integer to encode the padding.

```
typedef struct Stack_Allocation_Header S
struct Stack_Allocation_Header {
    uint8_t padding;
};
```

This padding stores the amount of bytes that has to be placed before the header in order to have the new allocation correctly aligned.





Note: Storing the padding as a byte does limit the the maximum alignment that can used with this stack allocator to 128 bytes. If you require a higher alignment, increase the size of integer used to store the padding. To calculate the maximum alignment that the padding can be used for, use this equation:

Maximum Alignment in Bytes = $2^{8 \times \text{sizeof(padding)} - 1}$

Init

The `stack_init` procedure just initializes the parameters for the given stack.

```
void stack_init(Stack *s, void *backing_
    s->buf = (unsigned char *)backing_bu
    s->buf_len = backing_buffer_length;
    s->offset = 0;
}
```

Alloc

Unlike an arena, a stack allocator requires a header alongside the allocation. As previously stated above, the `calc_padding_with_header` procedure is similar to the `align_forward` procedure from the previous article, however it determines how much space is needed with respect to the header and the re-

requested alignment. In the header, the amount padding needs to be stored and the address after that header needs to be returned.

```

size_t calc_padding_with_header(uintptr_t
    uintptr_t p, a, modulo, padding, need

    assert(is_power_of_two(alignment));

    p = ptr;
    a = alignment;
    modulo = p & (a-1); // (p % a) as it

    padding = 0;
    needed_space = 0;

    if (modulo != 0) { // Same logic as
        padding = a - modulo;
    }

    needed_space = (uintptr_t)header_size;

    if (padding < needed_space) {
        needed_space -= padding;

        if ((needed_space & (a-1)) != 0)
            padding += a * (1+(needed_sp
        } else {
            padding += a * (needed_space
        }
    }

    return (size_t)padding;
}

void *stack_alloc_align(Stack *s, size_t
    uintptr_t curr_addr, next_addr;
    size_t padding;
    Stack_Allocation_Header *header;

    assert(is_power_of_two(alignment));

    if (alignment > 128) {
        // As the padding is 8 bits (1 b
        // be used is 128 bytes
    }

```

```

        alignment = 128;
    }

    curr_addr = (uintptr_t)s->buf + (uintptr_t)padding;
    padding = calc_padding_with_header(curr_addr, size, alignment);
    if (s->offset + padding + size > s->end)
        // Stack allocator is out of memory
        return NULL;
    }
    s->offset += padding;

    next_addr = curr_addr + (uintptr_t)padding;
    header = (Stack_Allocation_Header *)next_addr;
    header->padding = (uint8_t)padding;

    s->offset += size;

    return memset((void *)next_addr, 0, size);
}

// Because C does not have default parameters
void *stack_alloc(Stack *s, size_t size)
    return stack_alloc_align(s, size, DEFAULT_ALIGNMENT);
}

```

Free

For `stack_free`, the pointer passed needs to be checked as to whether it is valid (i.e. it was allocated by this allocator). If it is valid, this means it is possible to acquire the header of this allocation. Using a little *pointer arithmetic*, we can reset the offset to the allocation previous to the passed pointer.

```

void stack_free(Stack *s, void *ptr) {
    if (ptr != NULL) {
        uintptr_t start, end, curr_addr;
        Stack_Allocation_Header *header;
        size_t prev_offset;

        start = (uintptr_t)s->buf;
    }
}

```



```

        end = start + (uintptr_t)s->buf_
        curr_addr = (uintptr_t)ptr;

        if (!(start ≤ curr_addr && curr_
            assert(0 && "Out of bounds m
            return;
        }

        if curr_addr ≥ start+(uintptr_t
            // Allow double frees
            return;
        }

        header = (Stack_Allocation_Heade
        prev_offset = (size_t)(curr_addr

        s->offset = prev_offset;
    }
}

```

Resize

Resizing an allocation is sometimes useful in an stack allocator. As we don't store the previous offset for this particular version, we will just reallocate new memory if there needs to be a change in allocation size².

```

void *stack_resize_align(Stack *s, void
    if (ptr == NULL) {
        return stack_alloc_align(s, new_
    } else if (new_size == 0) {
        stack_free(s, ptr);
        return NULL;
    } else {
        uintptr_t start, end, curr_addr;
        size_t min_size = old_size < new
        void *new_ptr;

        start = (uintptr_t)s->buf;
        end = start + (uintptr_t)s->buf_
        curr_addr = (uintptr_t)ptr;
        if (!(start ≤ curr_addr && curr_
            assert(0 && "Out of bounds m

```

² It is an exercise for the reader to figure out how to make this more efficient by not allocating more memory if it is was the previous allocation.

```

        return NULL;
    }

    if (curr_addr ≥ start + (uintpt
        // Treat as a double free
        return NULL;
    }

    if old_size == size {
        return ptr;
    }

    new_ptr = stack_alloc_align(s, n
    memmove(new_ptr, ptr, min_size);
    return new_ptr;
}

void *stack_resize(Stack *s, void *ptr, s
    return stack_resize_align(s, ptr, ol
}

```

Free All

Finally, `stack_free_all` is used to free all the memory within the allocator by setting the buffer offsets to zero. This is very useful for when you want to reset on a per cycle/frame basis. This acts identically to an arena in this case.

```

void stack_free_all(Stack *s) {
    s->offset = 0;
}

```

Improving the Stack Allocator

This loose/small stack allocator above is already very useful but it does not enforce the LIFO principle

for frees. It allows the user to free any block of memory in any order but frees everything that was allocated after it. In order to enforce the LIFO principle, data about the previous offset needs to be stored in the header and the general data structure.

```
struct Stack_Allocation_Header {
    size_t prev_offset;
    size_t padding;
};

struct Stack {
    unsigned char *buf;
    size_t buf_len;
    size_t prev_offset;
    size_t curr_offset;
};
```

This new header is a lot larger compared to the simple padding approach³, but it does mean that LIFO for frees can be enforced. There only needs to be a few adjustments to the code. The resize procedure is left as an exercise for the reader.

³ You can reduce the size of the header by using smaller integers but this does reduce the size of the allocations that can be used.

stack_alloc_align

```
...

s->prev_offset = s->offset; // Store the
s->offset += padding;

next_addr = curr_addr + (uintptr_t)padding;
header = (Stack_Allocation_Header*)(next_addr);
header->padding = padding;
header->prev_offset = s->prev_offset; //

s->offset += size;
```

stack_free

```
...

header = (Stack_Allocation_Header *)(curr

// Calculate previous offset from the he
prev_offset = (size_t)(curr_addr - (uint

if (prev_offset != header->prev_offset)
    assert(0 && "Out of order stack allo
    return;
}

// Reset the offsets to the previous all
s->curr_offset = s->prev_offset;
s->prev_offset = header->prev_offset;
```

Comments and Conclusion

The stack allocator is the first of many allocators that will use the concept of a *header* for allocations. In this basic form of a stack allocator, unless you want the LIFO behaviour enforced, I would personally recommend using an arena allocator with the `Temp_Arena_Memory` construct instead. However if you require something like C++'s constructors and destructors, a stack allocator will be more friendly to that framework (RAII) ⁴.

You can extend the stack allocator even further by having two different offsets: one that starts at the beginning and increments forwards,

⁴ https://wikipedia.org/wiki/Placement_syntax

and another that starts at the end and increments backwards. This is called a double-ended stack and allows for the maximization of memory usage whilst keeping fragmentation extremely low (as long as the offset never overlap).

In the next article, I will discuss *pool allocators* and how they are extremely useful for creating and destroying things in completely random order of the same size.

© 2007-2026 Ginger Bill